
ADPLL DESIGN DOCUMENT

---

1. Introduction

This document describes the architecture, design, and verification of our fully digital Phase-Locked Loop (ADPLL) implemented using verilog for TinyTapeout. The ADPLL synchronizes a digitally controlled oscillator (DCO) to an external reference clock using only synchronous digital logic. All internal logic runs on a 50 MHz system clock.

The design supports:
- Locking to a 1 MHz reference
- Tolerance to jitter
- Re-locking after frequency changes
- Numerically controlled oscillator with high resolution
- Digital lock detection
- Full cocotb verification in Icarus Verilog

---

2. System Overview

The ADPLL consists of four modules:

1. Phase Detector
2. Loop Filter (PI controller)
3. Digitally Controlled Oscillator (DCO)
4. Lock Detector and Top-Level Control

Data flow diagram (plain-text):

ref_signal → Phase Detector → Loop Filter → DCO Control Word → DCO → dco_signal
↑
└── Lock Detector ──────────┘

The loop attempts to minimize phase difference between the reference clock and the DCO output.

---

3. Module Descriptions and Signal Tables

Each module contains a table describing inputs, outputs, wires, registers, parameters, and functional purpose.

3.1 Phase Detector (phase_detector.v)

Purpose:
Determines whether the reference clock edge arrives before or after the DCO clock edge. Produces a “bang-bang” phase error of +1, -1, or 0.

Inputs:
- clk – 50 MHz system clock
- reset_n – active-low reset
- ref_in – asynchronous reference clock
- dco_in – DCO feedback clock

Outputs:
- phase_err – signed integer (+ERR_STEP or -ERR_STEP)
- edge_valid – high for one cycle when an edge (ref or dco) is detected

Internal Registers:
- ref_ff1, ref_ff2 – two-stage synchronizer for ref_in
- ref_prev – previous synchronized ref value
- dco_ff, dco_prev – registered DCO signal
- phase_err – actual stored error
- edge_valid – indicates detection of a rising edge

Operation Summary:
The module synchronizes ref_in, detects rising edges on both ref and dco, and outputs +ERR_STEP if the reference clock leads or -ERR_STEP if the DCO leads. If both rise on the same cycle or neither rises, error is 0.

3.2 Loop Filter (loop_filter.v)

Purpose:
Implements a digital Proportional + Integral controller. Converts the discrete bang-bang error into a smooth signed control value.

Parameters:
- K_P – proportional gain (default 4)
- I_SHIFT – integral gain factor (integrator adds phase_err divided by 2^I_SHIFT)
- IN_WIDTH – width of phase error
- OUT_WIDTH – control word width

Inputs:
- clk
- reset_n
- update_en – triggered by edge_valid from PD
- phase_in – signed phase error

Output:
- control_out – signed PI-controlled output to DCO

Internal Registers:
- acc – integral accumulator
- control_out – output word combining proportional and integral parts

Operation Summary:
Each time update_en is high (on a reference or DCO edge), the loop filter updates. The proportional term reacts immediately to error. The integral term accumulates small errors and eliminates long-term frequency offset. The result is a stable correction word sent to the DCO.

3.3 Digitally Controlled Oscillator (dco.v)

Purpose:
Implements a Numerically Controlled Oscillator (NCO). Generates a square wave whose frequency depends on ctrl_word.

Parameters:
- CTRL_BITS – width of ctrl_word
- PHASE_BITS – width of internal phase accumulator

Inputs:
- clk – 50 MHz
- reset_n
- ctrl_word – unsigned frequency increment

Output:
- dco_out – MSB of phase accumulator toggles as the DCO output

Internal Registers:
- phase – the NCO accumulator

Operation Summary:
Each clock cycle the phase accumulator adds ctrl_word. The output dco_out is taken from the most significant bit. The resulting frequency is:
(f_clk * ctrl_word) / (2^PHASE_BITS)

Example:
With 50 MHz clock and 24-bit accumulator, a ctrl_word of 335,544 produces approximately a 1 MHz output.

3.4 Top-Level Control (tt_um_richad.v)

Purpose:
Integrates all modules, maps PI output to DCO control, applies saturation, and implements a digital lock detector.

Inputs:
- clk – 50 MHz
- rst_n
- ena
- ui_in[0] = reference clock

Outputs:
- uo_out[0] – lock indicator

• uo_out[1] – dco_signal
• remaining 6 bits zero

Internal Signals:
• ref_signal – ui_in[0]
• phase_err – from PD
• edge_valid – from PD
• control_word – from Loop Filter
• dco_ctrl – saturated DCO tuning word
• dco_signal – from DCO
• lock_cnt – counts “good” edges
• lock_reg – final lock state

Lock Detector Operation:
The control_word is converted into a ctrl_delta and added to a nominal tuning value DCO_BASE.
If dco_ctrl is within the window (DCO_BASE ± 64), lock_cnt increments.
If outside, lock_cnt decrements.
Lock is declared when lock_cnt reaches or exceeds 64.

---

4. Loop Behavior

The PLL operates in three phases:

1. Acquisition (0–15 microseconds)
   • DCO frequency differs from reference
   • phase_err biased strongly positive or negative
   • control word ramps significantly

2. Convergence (15–30 microseconds)
   • DCO frequency now matches reference frequency
   • edges no longer drift
   • phase_err alternates ±1
   • control word oscillates around DCO_BASE

3. Lock (after ~30 microseconds)
   • bounded phase difference
   • frequencies identical
   • lock_cnt rises above threshold
   • lock_reg becomes 1

Because the PLL uses a bang-bang phase detector, a small, constant phase difference and jitter around the ideal value is expected and represents correct behavior.

---

5. Limitations

1. Uses a bang-bang phase detector, which introduces limit-cycle jitter.

2. Loop gains (K_P = 4, I_SHIFT = 4) are tuned for approximately 1 MHz reference.
   Effective lock range is about 0.5 MHz to 2 MHz.

3. Lock detector assumes nominal tuning near DCO_BASE for 1 MHz.

4. System clock (50 MHz) limits timing resolution to 20 ns.

5. Maximum DCO frequency is approximately 3.125 MHz with current PHASE_BITS.

---

6. Applications

Suitable applications:
- Low-power digital clock recovery
- On-chip timing alignment
- Simple frequency synthesis
- Educational ASIC projects
- Basic digital communication modulation experiments

---

7. Verification Through Cocotb

Four cocotb tests were used:

1. Basic Lock Test
   - Clean 1 MHz reference

    - Confirms lock within 2 ms

2. Jitter Test
    - Adds ±2 ns jitter
    - Confirms robustness

3. Frequency Step Test
    - Start at 0.9 MHz, re-lock
    - Jump to 1.1 MHz, re-lock
    - Demonstrates frequency tracking stability

4. Lock Observation Test
    - Runs approximately 150 microseconds
    - Used to generate waveform
    - Shows initial drift, convergence, control stabilization, and lock flag assertion

Cocotb logs confirm lock time of approximately 30–35 microseconds on all tests.

---

8. Waveform Interpretation

During the observation test:

Before Lock:
- DCO edges drift relative to reference
- phase_err biased strongly + or -
- control word ramps

During Convergence:
- DCO and reference have same period
- small phase oscillations occur
- dco_ctrl oscillates in tight range near DCO_BASE

Locked:
- No frequency drift
- Phase oscillates within a small bound
- lock_cnt reaches threshold
- lock_reg asserted

1) test_lock_observation (waveform-focused)

Suggested time windows:

Test: test_lock_observation (1 MHz clean reference, runs ~150 µs)

1. 0 – 10 µs
   • Startup / acquisition
   • DCO frequency clearly different from reference
   • DCO edges drifting relative to ref
   • phase_err mostly one sign (+ or –)
   • dco_ctrl ramping in one direction

2. 10 – 25 µs
   • Convergence region
   • DCO period very close to ref period
   • Edges no longer drifting; offset changing slowly
   • phase_err starts to alternate +1 / –1 instead of long runs
   • dco_ctrl stops ramping and begins to hover

3. 25 – 35 µs
   • Lock formation
   • DCO and ref have same frequency (no long-term drift)
   • Small, bounded phase offset (a few tens of ns)
   • phase_err mostly ±1, balanced
   • dco_ctrl sitting near DCO_BASE with small dither
   • lock_cnt climbing; lock_reg goes from 0 → 1 somewhere in this window

4. 35 – 150 µs
   • Steady locked behavior
   • DCO and ref perfectly frequency-aligned
   • phase_err continues to toggle ±1 with no bias
   • dco_ctrl stays inside lock window
   • lock_reg = 1 and remains high

---

2) test_basic_lock

Test: test_basic_lock (1 MHz clean reference, ends at ~32.5 µs)

1. 0 – 5 µs
   • Initial acquisition
   • DCO period noticeably wrong
   • phase_err biased strongly one way

- dco_ctrl ramping

2. 5 – 20 µs
   - Strong convergence
   - DCO period matches ref more closely
   - Edge drift slows and then stops
   - dco_ctrl approaches DCO_BASE and starts to wobble around it

3. 20 – 32 µs (near end of sim)
   - Just before and at lock
   - DCO and ref same frequency, bounded phase error
   - phase_err alternating ±1 (typical bang-bang lock)
   - dco_ctrl in lock window (DCO_BASE ± 64)
   - lock_cnt reaches threshold, lock_reg flips to 1 shortly before the test ends

---

3) test_jitter_lock

Test: test_jitter_lock (1 MHz reference with jitter, ends at ~32.5 µs)

1. 0 – 10 µs
   - Acquisition under jitter
   - DCO still far from ref, but edges are noisy due to jitter
   - phase_err mostly one sign but with some jitter-induced variation
   - dco_ctrl ramping in a noisy fashion

2. 10 – 25 µs
   - Convergence under jitter
   - DCO period matches the *average* reference period
   - Edges wander slightly because of jitter but no systematic drift
   - phase_err alternates ±1 with more randomness
   - dco_ctrl oscillates but stays near a mean value

3. 25 – 32 µs
   - Locked with jitter
   - DCO follows jittered ref without long-term phase walk-off
   - phase_err looks noisy but statistically balanced
   - dco_ctrl remains inside lock window most of the time
   - lock_reg becomes 1 and stays 1 despite jitter

---

4) test_freq_step

Test: test_freq_step (0.9 MHz → 1.1 MHz step, ends at ~36 µs)

Your test does:

- Start at 0.9 MHz and lock
- Then switch to 1.1 MHz and re-lock

The exact switch time is defined in the test when the first lock is seen, but the behavior breaks down nicely like this:

1. 0 – 15 µs (lock to 0.9 MHz)
   • Startup and first acquisition
   • DCO initially off, then converges to 0.9 MHz
   • phase_err biased at first, then starts alternating
   • dco_ctrl settles to a value **below** the 1 MHz DCO_BASE (lower increment → lower frequency)
   • lock_reg goes high once 0.9 MHz lock is achieved

2. Around 15 – 20 µs (frequency step moment)
   • Reference switches from 0.9 MHz to 1.1 MHz
   • Suddenly, ref edges arrive earlier than expected
   • phase_err becomes biased (indicating DCO now “slow” compared to new ref)
   • dco_ctrl ramps upward to increase DCO frequency

3. 20 – 30 µs (re-lock to 1.1 MHz)
   • DCO frequency converges to new 1.1 MHz reference
   • phase_err returns to balanced ±1 behavior
   • dco_ctrl settles to a value **above** the 1 MHz DCO_BASE (higher increment)
   • lock_reg goes low briefly (depending on window), then high again once control is stable

4. 30 – 36 µs (final locked state)
   • DCO and ref at 1.1 MHz, no long-term drift
   • phase_err bounded
   • dco_ctrl stable in new band
   • lock_reg = 1 at end of test